

Task 15: Pandas DataFrame Basics

Task focus: Create DataFrames from dictionaries and datasets and explore rows, columns, shape, data types, and summary information.

1. Task Overview

The purpose of this task is to understand the Pandas **DataFrame**. A DataFrame is the main Pandas structure for working with tabular data. It organizes information into labeled rows and columns and is widely used for data cleaning, exploration, analysis, and preparation.

2. Objective

Learn the primary Pandas structure used for tabular data analysis. By the end of the task, you should be able to create a DataFrame, load a dataset, inspect its structure, select useful data, filter rows, and generate basic statistical summaries.

3. Tools

Python: programming language used for the implementation.

Pandas: library used for tabular data manipulation and analysis.

Jupyter Notebook: interactive environment for executing and documenting Python code.

4. What is a DataFrame?

A DataFrame is a **two-dimensional labeled data structure**. It has rows and columns, an index, column names, and a data type for each column. For example, a student table may contain Name, Age, Marks, and Attendance as columns.

5. Why DataFrames are Important

DataFrames make it easier to inspect and manipulate structured data. They provide convenient operations for selecting columns, filtering rows, calculating statistics, detecting missing values, sorting data, and preparing datasets for later analysis or machine learning.

6. Creating a DataFrame Manually

A DataFrame can be created with **pd.DataFrame()**. A common approach is to provide a dictionary in which each key becomes a column name and each list supplies that column's values.

7. Loading a Dataset

For a CSV file, use **pd.read_csv()**. This converts the tabular file into a Pandas DataFrame that can be explored and analyzed using Pandas methods.

8. Inspecting the First and Last Rows

head() displays the first rows and is useful for checking whether the dataset loaded correctly. **tail()** displays the last rows and can help verify the end of the dataset.

9. Understanding Shape

The **shape** attribute returns a tuple in the form (**rows, columns**). If a DataFrame has shape (8, 8), it contains 8 rows and 8 columns.

10. Understanding Columns

The **columns** attribute provides the column labels. You can use these names to select the information needed for an analysis.

11. Understanding Data Types

The **dtypes** attribute shows the data type of each column. Numerical columns can be used for mathematical calculations, while text columns are generally used for labels or categories.

12. Using info()

The **info()** method provides a compact structural overview. It helps identify the number of rows, column names, non-null counts, and data types. This is one of the first checks to perform after loading a dataset.

13. Using describe()

The **describe()** method summarizes numerical columns. It commonly reports count, mean, standard deviation, minimum, quartiles, and maximum. These statistics provide a quick understanding of the distribution and range of numerical data.

14. Selecting One or Multiple Columns

For one column, use **df['Column']**. For multiple columns, use **df[['Column1', 'Column2']]**. Selecting only required columns makes analysis more focused and readable.

15. Filtering Rows

Boolean filtering allows you to keep rows that satisfy a condition. For example, **df[df['Average'] >= 80]** returns students whose average is at least 80. For multiple conditions, use **&** for AND and **|** for OR, with each condition inside parentheses.

16. Checking Missing Values

Use **df.isnull().sum()** to count missing values in each column. Missing values should be identified before analysis because they can affect calculations and conclusions.

17. Practical Student Dataset

The included dataset contains Student_ID, Name, Age, Python_Marks, SQL_Marks, Attendance, Average, and Performance. It demonstrates how a DataFrame can support realistic student-performance analysis.

18. Practical Analysis

The project calculates average Python marks, finds the highest SQL marks, identifies the lowest attendance, filters students with high average scores, selects relevant columns, and checks the dataset for missing values.

19. Efficient DataFrame Workflow

A clear workflow is: **Load** → **Inspect** → **Validate** → **Select** → **Filter** → **Summarize** → **Interpret**. First load the data, inspect its structure, validate data types and missing values, select relevant fields, filter records when needed, calculate summaries, and finally interpret the results.

20. DataFrame vs Series

A **Series** is one-dimensional labeled data, while a **DataFrame** is two-dimensional tabular data. A DataFrame can contain multiple Series as columns. This distinction is important when moving from Task 14 (Series) to Task 15 (DataFrame).

21. Common Mistakes

Common mistakes include using an incorrect column name, forgetting parentheses around multiple Boolean conditions, confusing **shape** with **size**, ignoring missing values, and attempting numerical calculations on inappropriate data types.

22. Best Practices

Use clear variable and column names, inspect data before analysis, check data types and missing values, select only required columns, prefer vectorized Pandas operations, and keep analysis steps organized and reproducible.

23. Complete Practical Python Program

```
import pandas as pd

# 1. Create a DataFrame manually
student_df = pd.DataFrame({
    "Name": ["Aman", "Priya", "Rahul", "Neha"],
    "Age": [20, 21, 20, 22],
    "Marks": [82, 91, 76, 88]
})
print("MANUAL DATAFRAME")
print(student_df)

# 2. Load the real dataset
df = pd.read_csv("student_performance.csv")

# 3. Inspect the dataset
print("\nFIRST FIVE ROWS")
print(df.head())

print("\nLAST FIVE ROWS")
print(df.tail())

print("\nSHAPE:", df.shape)
print("ROWS:", df.shape[0])
print("COLUMNS:", df.shape[1])

print("\nCOLUMN NAMES")
print(df.columns.tolist())

print("\nDATA TYPES")
print(df.dtypes)

print("\nDATA INFORMATION")
df.info()

# 4. Summary statistics
print("\nSUMMARY STATISTICS")
print(df.describe())

# 5. Practical analysis
print("\nAVERAGE PYTHON MARKS:", round(df["Python_Marks"].mean(), 2))
print("HIGHEST SQL MARKS:", df["SQL_Marks"].max())
print("LOWEST ATTENDANCE:", df["Attendance"].min())

# 6. Filtering
print("\nSTUDENTS WITH AVERAGE >= 80")
print(df[df["Average"] >= 80][["Name", "Average", "Performance"]])

# 7. Missing values
print("\nMISSING VALUES")
print(df.isnull().sum())
```

24. Step-by-Step Explanation of the Program

Step 1: `pd.DataFrame()` creates a small student DataFrame manually.

Step 2: `pd.read_csv()` loads the prepared student dataset.

Step 3: `head()`, `tail()`, `shape`, `columns`, `dtypes`, and `info()` explore the structure.

Step 4: `describe()` generates numerical summary statistics.

Step 5: `mean()`, `max()`, and `min()` calculate useful performance measures.

Step 6: Boolean filtering selects students with `Average >= 80`.

Step 7: `isnull().sum()` checks data quality.

25. Interview Questions and Answers

Q: What is a Pandas DataFrame?

A: A two-dimensional labeled data structure used to store, manipulate, and analyze tabular data.

Q: How do you create a DataFrame?

A: Use `pd.DataFrame()`, commonly with a dictionary, list of dictionaries, array, or another structured source.

Q: How do you load a CSV file?

A: Use `pd.read_csv('filename.csv')`.

Q: What does `head()` do?

A: It displays the first rows of a DataFrame and is useful for quickly inspecting the loaded data.

Q: What does `info()` provide?

A: It provides a structural summary including column names, non-null counts, data types, and other DataFrame information.

Q: What is the difference between `shape` and `size`?

A: `shape` returns the dimensions as (rows, columns), while `size` returns the total number of elements.

Q: What does `describe()` do?

A: It provides descriptive statistics for numerical columns, such as count, mean, standard deviation, quartiles, minimum, and maximum.

Q: How do you select multiple columns?

A: Use a list of column names, for example `df[['Name', 'Average']]`.

Q: How do you filter rows?

A: Use a Boolean condition, for example `df[df['Average'] >= 80]`.

26. Practice Questions

1. Create an employee DataFrame using a dictionary.
2. Load a CSV file into a DataFrame.
3. Display the first 10 rows.
4. Find the shape and column names.
5. Display the data types.
6. Generate descriptive statistics.
7. Filter records above a chosen score or salary.
8. Check missing values in all columns.

27. Top 5 Skills

1. Pandas
2. DataFrame Data Manipulation
3. Data Inspection and Exploration
4. Data Filtering and Selection
5. Python Data Analysis

28. Feedback

The task was very useful for understanding Pandas DataFrames and their role in tabular data analysis. I learned how to create DataFrames, load datasets, inspect rows and columns, check shape and data types, generate summary statistics, select columns, filter records, and check missing values.